1 Derived rules and positive-depth powers

Source: `PrimeTensor/Analysis/Examples.lean`

What this file establishes

Nothing in this file is new analysis, and the module header says so: it develops the consequences that require no further continuity assumption. Three derivatives are computed — of a ratio, of the reciprocal map, and of every positive-depth power — and all three are assembled from the product and inversion rules of `PrimeTensor/Analysis/Rules.lean` and the two exact examples of `PrimeTensor/Analysis/Derivative.lean`.

The substantive content is the power operation. `PrimeTensor/Structure.lean` rejected Lean's CommGroup partly because it would equip the carrier with a natural-power field defined at 0; this file supplies the replacement, a power indexed by Depth with no zeroth case. The power rule that results, Theorem 1.10, therefore has — in the docstring's words — no zeroth case to prove or even state.

1.1 Positive-depth powers

Definition 1.1 (`MulReal.depthPow`)**.** For a : MulReal, define $\mathrm{depthPow}(a, -)$: Depth $\to$ MulReal by

$$\mathrm{depthPow}(a, \mathsf{one}) = a, \qquad \mathrm{depthPow}(a, \mathsf{succ}\, d) = a \cdot \mathrm{depthPow}(a, d),$$

so that $\mathrm{depthPow}(a, d) = a^{|d|}$. Two simp lemmas, depthPow_one and depthPow_succ, record the two equations, both by rfl.

```
def depthPow (a : MulReal) : Depth → MulReal
  | .one => a
  | .succ d => a * depthPow a d
```

Remark 1.2 (Two Depth-indexed iterations, doing different jobs). This is the second operation in the development defined by recursion on a depth, and it is worth setting beside the first. scalePow of `PrimeTensor/Scale.lean` *squares* at each successor and so reaches exponent $2^{|d|-1}$ in $|d|$ steps; depthPow multiplies by one more copy and so reaches exponent $|d|$ in $|d|$ steps. The first climbs geometrically because it is measuring tolerances, which have to become fine quickly; the second climbs linearly because it is an exponent, which has to be able to take every positive value. The same index type is carrying two quite different scales, and only the linear one is a power in the ordinary sense.

Design note 1.3 (The zeroth power, promised absent and now absent). `PrimeTensor/Structure.lean` declined CommGroup on the ground that Monoid carries an npow field defined at 0, which would reintroduce at the level of the carrier the zeroth index that `PrimeTensor/Depth.lean` had excluded. Declining a library operation is only half a decision; this file completes it by defining the operation the development actually wants.

The consequence is visible in Theorem 1.10. An induction over $\mathbb{N}$ would have a base case $a^0 = 1$ with derivative trivial, and the statement would have to be checked there. The induction here starts at one, where the power is a itself and the derivative is identity — a case that carries content rather than being a degenerate convention. Nothing is lost: every exponent the additive version could state, except 0, is a depth.

1.2 Derived tangent maps

Definition 1.4 (`MulTangentMap.ratio`)**.** $\mathrm{ratio}(D, E) \equiv \mathrm{mul}(D, \mathrm{inv}(E))$, with $\mathrm{ratio}(D, E)(h) = D(h)\,E(h)^{-1}$ by rfl.

No verification is needed: the two operations of `PrimeTensor/Analysis/Rules.lean` already produce tangent maps, so their composite does too. As the docstring notes, the ratio is built only from product and inversion, introducing no third primitive on tangent maps — the same discipline `PrimeTensor/Structure.lean` applied in refusing Div on the carrier.

Definition 1.5 (`MulTangentMap.depthPow`)**.**

$$\mathrm{depthPow}(\mathsf{one}) = \mathrm{identity}, \qquad \mathrm{depthPow}(\mathsf{succ}\, d) = \mathrm{mul}(\mathrm{identity}, \mathrm{depthPow}(d)).$$

```
def depthPow : Depth → MulTangentMap
  | .one => identity
  | .succ d => mul identity (depthPow d)
```

Lemma 1.6 (`MulTangentMap.depthPow_apply`)**.** $\mathrm{depthPow}(d)(h) = \mathrm{depthPow}(h, d)$*: the d-th power tangent map sends a perturbation to its own d-th power.*

```
theorem depthPow_apply (d : Depth) (h : MulReal) :
    depthPow d h = MulReal.depthPow h d := by
  induction d with
  | one =>
      rfl
  | succ d ih =>
      change h * depthPow d h = h * MulReal.depthPow h d
      rw [ih]
```

Proof. By induction on d. At one both sides are h. At $\mathsf{succ}\, d$ both sides unfold to h times the respective value at d, and the induction hypothesis identifies those. □

Remark 1.7 (Stated but unused here). Lemma 1.6 is not cited by Theorem 1.10, whose proof unfolds both power operations definitionally with change and never needs the two to be identified as terms. The lemma is there to be quotable, and to record for the reader that the two recursions — one on the carrier, one on tangent maps — compute the same thing.

1.3 Three derivatives

Theorem 1.8 (`hasMulDerivativeAt_ratio`)**.** *If f has derivative D_f at x and g has derivative D_g at x, then $y \mapsto \mathrm{ratio}(f(y), g(y))$ has derivative $\mathrm{ratio}(D_f, D_g)$ at x.*

Proof. Unfold both ratios to a product with an inverse and compose the two rules of `PrimeTensor/Analysis/Rules.lean`. The proof is one line because both ratios were *defined* as that composite, on the carrier and on tangent maps alike; had either been a primitive, this would have been a third rule to prove rather than a corollary. □

Corollary 1.9 (`hasMulDerivativeAt_reciprocal`)**.** $y \mapsto y^{-1}$ *has derivative* $\mathrm{inv}(\mathrm{identity})$ *at every point.*

Proof. Apply the inversion rule to the identity derivative. □

Theorem 1.10 (`hasMulDerivativeAt_depthPow`)**.** *For every depth d and every x, the map $y \mapsto \mathrm{depthPow}(y, d)$ has derivative $\mathrm{depthPow}(d)$ at x.*

Proof. By structural recursion on d. At `one` the map is the identity and the claim is the identity derivative of `PrimeTensor/Analysis/Derivative.lean`. At $\mathsf{succ}\, d$ the map is $y \mapsto y \cdot \mathrm{depthPow}(y, d)$ and the model is $\mathrm{mul}(\mathrm{identity}, \mathrm{depthPow}(d))$, so the product rule applies to the identity derivative and the recursive call. □

Design note 1.11 (The derivative of a power does not depend on the point). Theorem 1.10 gives one tangent map that works at *every* x, and by Lemma 1.6 that map is $h \mapsto h^{|d|}$. The additive statement is $(y^n)' = n\, y^{n-1}$, which depends on y. The discrepancy is not an error on either side; it is the difference between two derivatives.

In the chart of `PrimeTensor/Analysis/Derivative.lean`, where both the argument and the value are read logarithmically, the quantity being computed is $\dfrac{d}{du}\Big[\log f(xe^u)\Big]_{u=0} = \dfrac{xf'(x)}{f(x)}$, the elasticity of f at x. For $f(y) = y^n$ that is the constant n, independent of x — which is exactly what the theorem says, since log of $h \mapsto h^n$ is multiplication by n.

The whole family computed so far reads off in the same currency: elasticity 1 for the identity, 0 for a constant, -1 for the reciprocal, n for the n-th power, the sum of the elasticities for a product, and their difference for a ratio. Every rule proved in `PrimeTensor/Analysis/Rules.lean` and in this file is the additivity of elasticity, which is why none of them carries a Leibniz term and why none of them mentions the point. Nothing in this paragraph is formalised.

Remark 1.12 (The negligibility machinery is still untouched). Every derivative known at the end of this file is obtained from identity and trivial by products and inverses, so the functions covered are exactly those of the form $y \mapsto c \cdot y^n$ with n an integer. For such a function the response is

$$\frac{c\,(x h_k)^n}{c\,x^n} = h_k^n,$$

which is the model *exactly*, with no error at all. So every derivative computed in the development so far routes through firstOrder_of_exact, and NegligibleRelative — defined in `PrimeTensor/Analysis/Derivative.lean` and equipped with closure properties in `PrimeTensor/Analysis/Rules.lean` — has still never been applied to a nonzero error. The first function needing it is not in this file.

Remark 1.13 (What is not proved). depthPow on the carrier is given no laws: neither $a^m a^n = a^{m+n}$, nor $(ab)^n = a^n b^n$, nor $(a^m)^n = a^{mn}$ is stated, and Depth carries no addition or multiplication with which to state the first and third. Negative powers exist only as $(a^n)^{-1}$, with no lemma identifying them with anything. There is still no chain rule and still no uniqueness of derivatives.

Summary of the correspondence

Lean declaration	Kind	Here
`MulReal.depthPow`	definition	Def. 1.1
`MulReal.depthPow_one`	simp thm	Def. 1.1
`MulReal.depthPow_succ`	simp thm	Def. 1.1
`MulTangentMap.ratio`	definition	Def. 1.4
`MulTangentMap.ratio_apply`	simp thm	Def. 1.4
`MulTangentMap.depthPow`	definition	Def. 1.5
`MulTangentMap.depthPow_one_apply`	simp thm	Def. 1.5
`MulTangentMap.depthPow_apply`	theorem	Lem. 1.6
`hasMulDerivativeAt_ratio`	theorem	Thm. 1.8
`hasMulDerivativeAt_reciprocal`	theorem	Cor. 1.9
`hasMulDerivativeAt_depthPow`	theorem	Thm. 1.10

Every declaration of `PrimeTensor/Analysis/Examples.lean` appears above. The file contains no `sorry`, no axiom, and no unproved named proposition. Design notes 1.3 and 1.11 and Remarks 1.2, 1.7, 1.12 and 1.13 are stated for the reader and are not declarations of the file; the elasticity reading is a gloss and is not formalised.